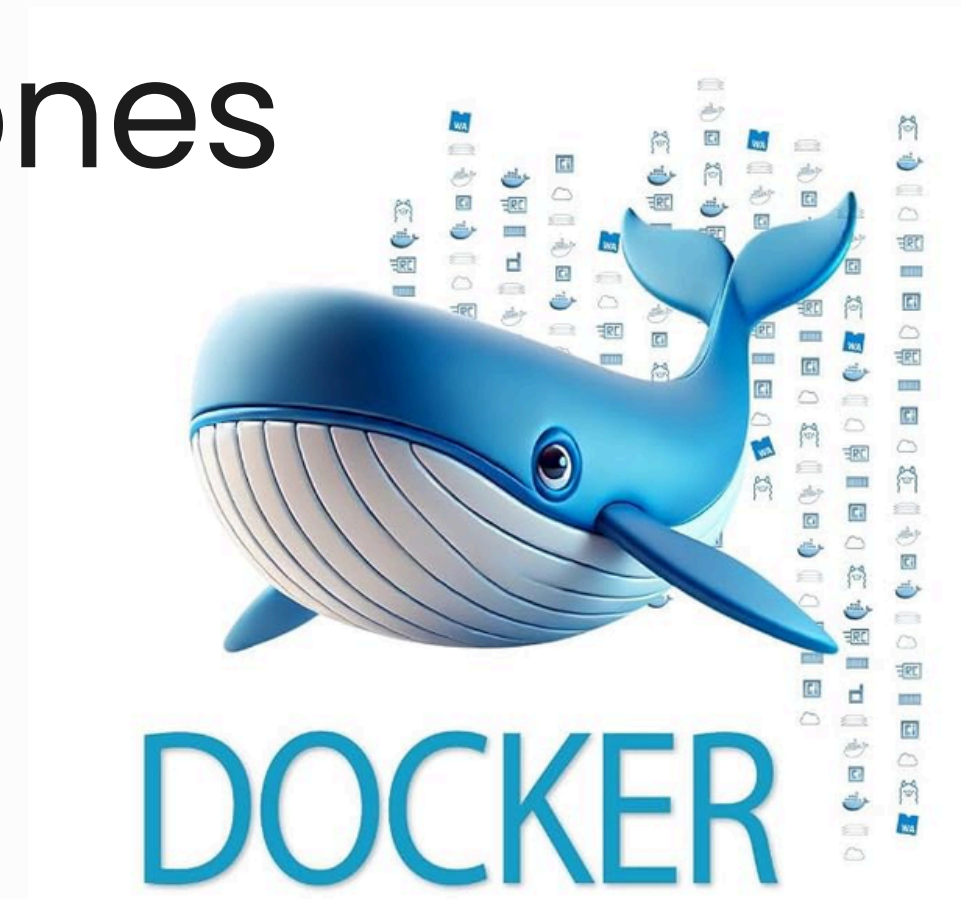


DOCKER DESDE CERO:

Crea y Despliega Aplicaciones

INSTRUCTOR: Cristian Jampier Chileno Segundo
Oficina de Tecnologías de la Información (OTI)
Programa Completo — PIT 2026



RESUMEN DE LA SESIÓN – 1

1. Una VM virtualiza una máquina completa mediante un hipervisor.
2. La instalación de Docker y Compose se valida con la documentación oficial.
3. La contenerización empaqueta aplicaciones y dependencias.
4. Docker permite ejecutar entornos reproducibles.
5. Una imagen es una plantilla; un contenedor es una ejecución de esa imagen.
6. Ya ejecutamos comandos base y una primera aplicación Flask en contenedor.

Ahora: Sesión 2

Dockerfile Profesional: instrucciones clave, capas, caché y buenas prácticas.

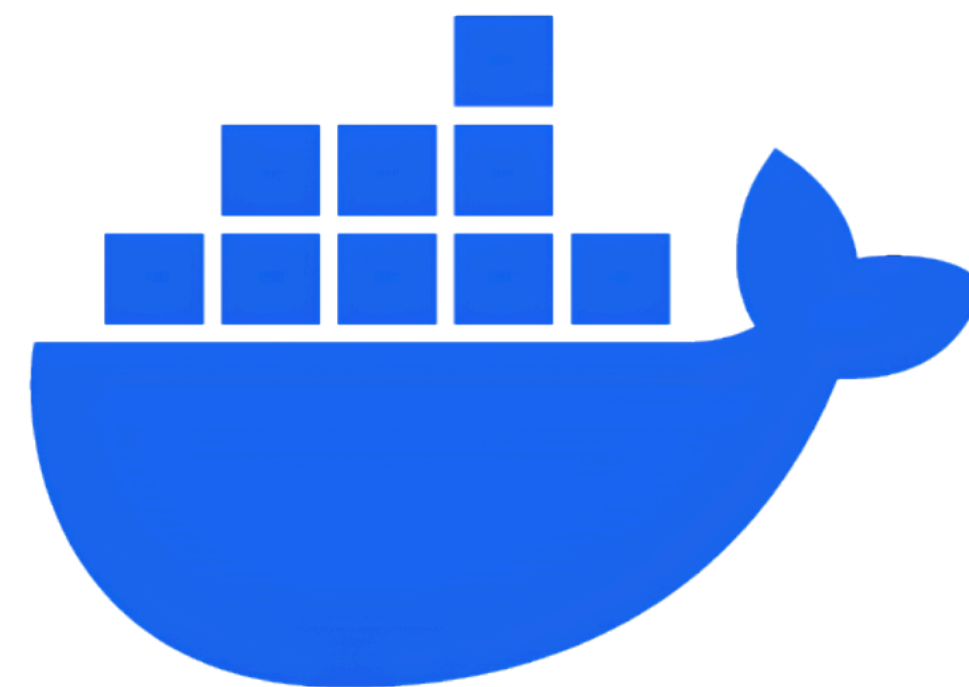
SESIÓN 2

- 1.Instrucciones Clave
- 2.Capas y Construcción
- 3.Optimización con Caché
- 4.Uso de .dockerignore
- 5.Imágenes Base Livianas
- 6.Publicación en Docker Hub

Objetivo de la sesión 2

Al terminar la sesión 2 podrás

- Escribir un Dockerfile con las instrucciones clave: **FROM, RUN, COPY, ENV, EXPOSE y CMD.**
- Explicar cómo funcionan las capas y cómo afectan el rendimiento del build.
- Aprovechar la caché de Docker para builds más rápidos.
- Usar .dockerignore para evitar archivos innecesarios en la imagen.
- Elegir imágenes base livianas (slim, alpine) según el caso.
- Publicar imágenes en Docker Hub con tagging y versionado.



De Dockerfile básico a imagen profesional

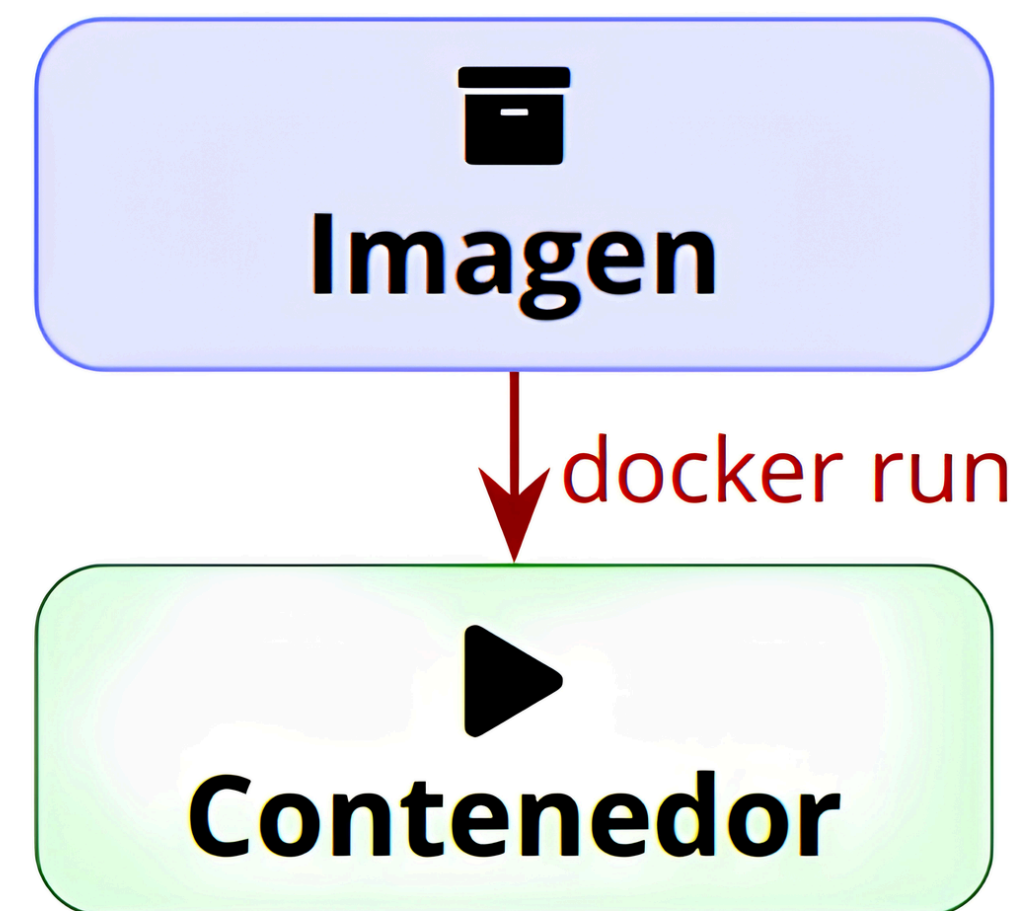
Recordatorio de la sesión anterior

Lo que ya dominamos

- Contenerización: empaquetar app + dependencias.
- Diferencia entre imagen (plantilla) y contenedor (ejecución).
- Docker resuelve el problema de «en mi máquina funciona».
- Comandos esenciales: run, ps, stop, rm, images.
- Construimos y ejecutamos una app Flask en contenedor.

Base para hoy

Sabemos ejecutar contenedores desde imágenes existentes. Ahora crearemos nuestras propias imágenes de forma profesional.



¿QUÉ ES UN DOCKERFILE?



Instrucciones clave del Dockerfile

FROM: EL PUNTO DE PARTIDA

```
1 FROM python :3.12 - slim
```

- Define la imagen base sobre la que se construye.
- Es la primera instrucción de todo Dockerfile.
- Puede ser una imagen oficial (python, node, nginx) o una imagen propia.

Consejo

Usa imágenes oficiales siempre que puedas. Son mantenidas, actualizadas y más seguras que imágenes de terceros.

Versiones

Evita :latest en producción. Fija una versión concreta (:3.12-slim) para builds reproducibles.

RUN: EJECUTAR COMANDOS DURANTE EL BUILD

```
1 RUN apt - get update && \  
2 apt - get install -y curl && \  
3 rm - rf / var / lib / apt / lists /*
```

- Ejecuta comandos durante la construcción de la imagen.
- Se usa para instalar paquetes, crear carpetas, configurar el sistema.
- Cada RUN crea una nueva capa en la imagen.

Buena práctica

Encadena comandos con && para reducir capas y limpiar cachés en la misma instrucción.

Regla de oro

Limpia archivos temporales (apt lists, pip cache) en el mismo RUN para no arrastrar basura a la imagen final.

COPY VS ADD

```
1 # COPY : copia archivos locales
2 COPY requirements .txt .
3 COPY app .py / app /
```

- Copia archivos o carpetas desde el contexto de build (tu PC) hacia la imagen.
- Es la opción recomendada para la mayoría de los casos.

```
1 # ADD: copia + descomprime + URLs
2 ADD app .tar .gz / app /
3 ADD https :// example . com / file / tmp /
```

- Además de copiar, descomprime automáticamente archivos .tar.gz.
- Puede descargar desde URLs.
- Úsalo solo cuando necesites la funcionalidad extra.

Regla práctica

Prefiere COPY. Usa ADD solo si necesitas descomprimir automáticamente

WORKDIR: EL DIRECTORIO DE TRABAJO

1 WORKDIR / app

- Establece el directorio de trabajo para las instrucciones siguientes.
- Si la carpeta no existe, Docker la crea automáticamente.
- Afecta a RUN, CMD, ENTRYPOINT, COPY y ADD.

¿Por qué usarlo?

- Evita rutas absolutas en cada comando.
- Hace el Dockerfile más legible.
- Previene errores de archivos en carpetas equivocadas.

Prefiere WORKDIR

Prefiere WORKDIR /app; RUN cd ... no persiste entre instrucciones.

ENV: VARIABLES DE ENTORNO

```
1 ENV FLASK_ENV = production
2 ENV APP_HOME = / app
3 ENV PYTHONUNBUFFERED =1
```

- Define variables de entorno disponibles en tiempo de build y en el contenedor.
- Ideal para configuraciones que no cambian entre entornos.
- Se pueden sobrescribir con `docker run -e`.

Cuándo usar ENV

- Configuración por defecto de la app.
- Versiones de herramientas.
- Rutas y paths estándar.

Cuándo NO

Secretos como contraseñas o tokens no deben quedar hardcodeados en la imagen.

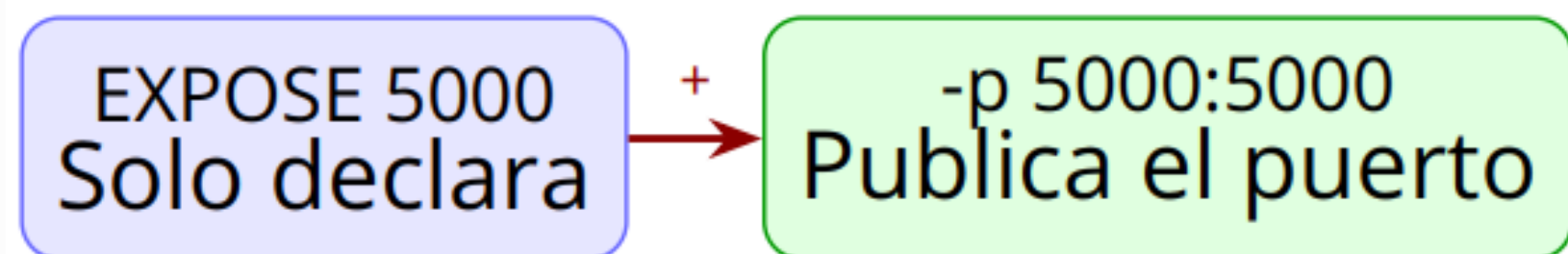
EXPOSE: DECLARAR PUERTOS

```
1 EXPOSE 5000  
2 EXPOSE 80/ tcp
```

- Documenta qué puertos escucha el contenedor.
- No publica el puerto automáticamente.
- Sirve como metadata para otros desarrolladores y herramientas.

EXPOSE vs -p

- EXPOSE: declara (informativo).
- docker run -p: publica y mapea el puerto.



CMD VS ENTRYPOINT

```
1 CMD ["python", "app.py"]
```

- Comando por defecto al iniciar el contenedor.
- Se puede sobrescribir desde docker run.
- Forma exec (array) recomendada sobre forma shell.

```
1 ENTRYPOINT ["python"]  
2 CMD ["app.py"]
```

- Define el ejecutable principal (no se sobrescribe fácilmente).
- CMD proporciona argumentos por defecto al ENTRYPOINT.
- Útil para crear herramientas CLI en contenedores.

Regla general

Usa CMD para apps. Usa ENTRYPOINT + CMD para herramientas CLI.

RESUMEN VISUAL DE INSTRUCCIONES

INSTRUCCIÓN	¿QUÉ HACE?	SOLUCIÓN RÁPIDA
FROM	Define la imagen base	Siempre, como primera línea
RUN	Ejecuta comandos en el build	Instalar dependencias, scripts de setup
COPY	Copia archivos locales a la imagen	Mover código y archivos del proyecto
WORKDIR	Establece el directorio de trabajo	Al inicio y cuando cambies de contexto
ENV	Define variables de entorno	Configuraciones por defecto
EXPOSE	Declara puertos de escucha	Documentar qué puertos usa la app
CMD	Comando por defecto del contenedor	Iniciar la aplicación

Capas, caché y buenas prácticas

¿CÓMO CONSTRUYE DOCKER UNA IMAGEN?

FROM python:3.12-slim

❖ *capa de la imagen base*

WORKDIR /app

❖ *capa 1*

COPY requirements.txt .

❖ *capa 2*

RUN pip install -r requirements.txt

❖ *capa 3*

COPY app.py .

❖ *capa 4*

CMD ["python", ".app.py"]

❖ *capa 5 (metadatos)*

Cada instrucción del Dockerfile genera una capa

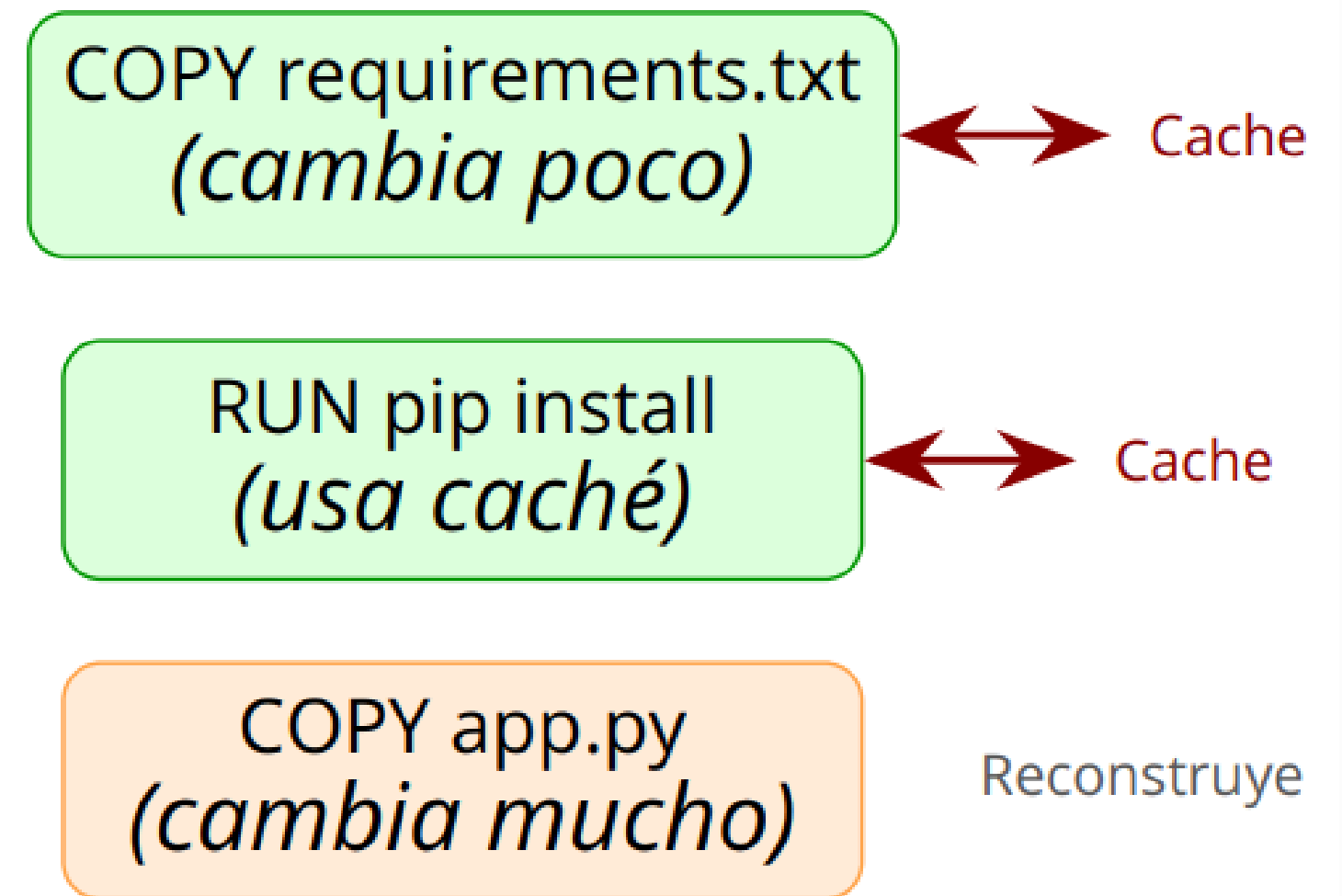
- Las capas son inmutables. Docker reutiliza las que no cambiaron mediante caché.

La caché de Docker: cómo funciona

- Docker compara cada instrucción con la caché de builds anteriores.
- Si la instrucción y sus dependencias no cambiaron, **reutiliza la capa.**
- Si una capa cambia, **todas las capas siguientes se reconstruyen.**

Regla de oro del orden

Pon primero lo que cambia menos (instalación de dependencias) y al final lo que cambia más (código de la app).



EJEMPLO: DOCKERFILE OPTIMIZADO

Mal orden (lento)

```
1 FROM python :3.12
2
3 COPY . / app # TODO cambia
4 WORKDIR / app
5
6 RUN pip install \ # Siempre
7 -r requirements . txt
8
9 CMD [" python "," app .py"]
```

Buen orden (rápido)

```
1 FROM python :3.12 - slim
2
3 WORKDIR / app
4
5 # Dependencias primero
6 COPY requirements . txt .
7 RUN pip install --no - cache - dir \
8 -r requirements . txt
9
10 # Código al final
11 COPY ..
12
13 CMD [" python "," app .py"]
```

¿Por qué es más rápido?

Cachea dependencias; cambios en app.py no reinstalan paquetes.

OTRAS BUENAS PRÁCTICAS

1. Usa imágenes base específicas

```
1 # Mal : imagen generica y pesada
2 FROM python
3
4 # Bien : version fija y liviana
5 FROM python :3.12 - slim
```

2. No instales paquetes innecesarios

```
1 RUN apt - get update && \
2 apt - get install -y --no - install - recommends \
3 curl && \
4 rm - rf / var / lib / apt / lists /*
```

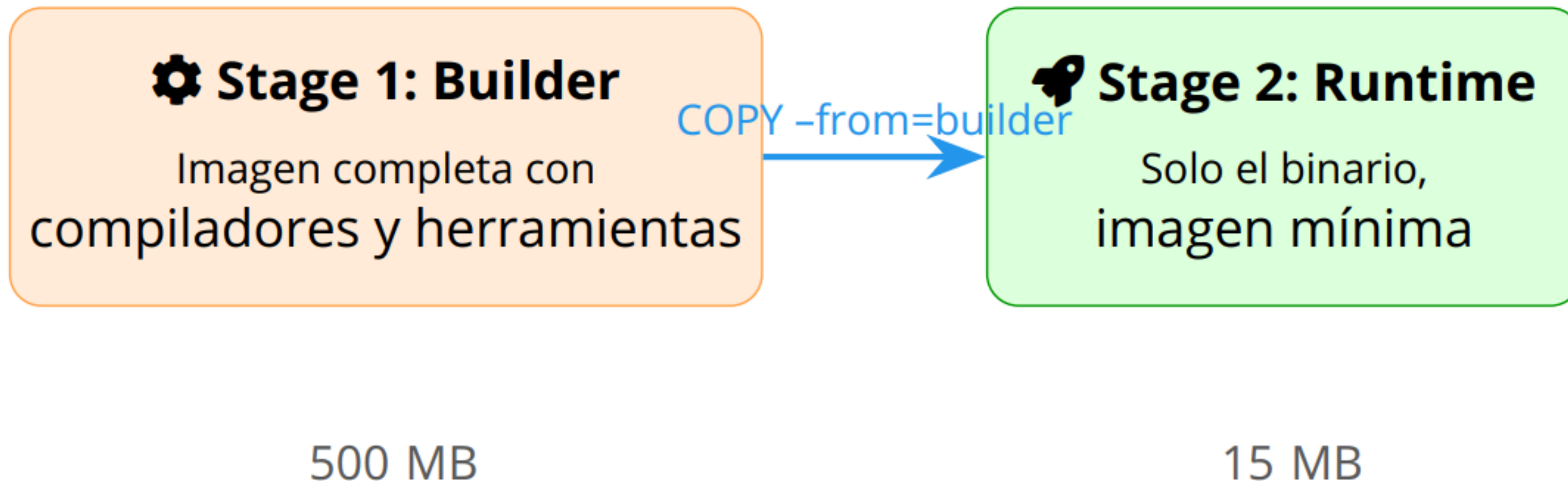
3. Limpia en el mismo RUN

```
1 RUN pip install -r requirements . txt && \
2 rm - rf / root /. cache / pip
```

4. Usa multistage builds

```
1 # Stage 1: build
2 FROM golang :1.21 AS builder
3 RUN go build -o app
4
5 # Stage 2: runtime ( imagen final pequeña)
6 FROM alpine :3.19
7 COPY -- from = builder / app .
8 CMD ["/ app "]
```

MULTISTAGE BUILD: EL PATRÓN PROFESIONAL



Ventaja

- La imagen final solo contiene lo necesario para ejecutar. Las herramientas de compilación quedan en la etapa de build y no contaminan la imagen de producción.

`.dockerignore` e imágenes base livianas

.DOCKERIGNORE: QUÉ ES Y PARA QUÉ SIRVE

```
# .dockerignore
__pycache__
*.pyc
*.pyo
.git
.env
*.log
node_modules
venv/
.venv/
.vscode/
*.md
Dockerfile
docker-compose.yml
```

- Similar a .gitignore: archivos y carpetas que no se envían al contexto de build.
- Reduce el tamaño del contexto y acelera el build.
- Evita que archivos sensibles (.env, claves) terminen en la imagen.

Regla

Incluye .dockerignore; evita enviar node_modules, .git y credenciales.

IMÁGENES BASE: SLIM VS ALPINE

Completa

`python:3.12`

1 GB

Todas las herramientas
Debian completo

Slim

`python:3.12-slim`

150 MB

Debian mínimo
Lo esencial para Python

Alpine

`python:3.12-alpine`

50 MB

Alpine Linux (musl)
La más liviana

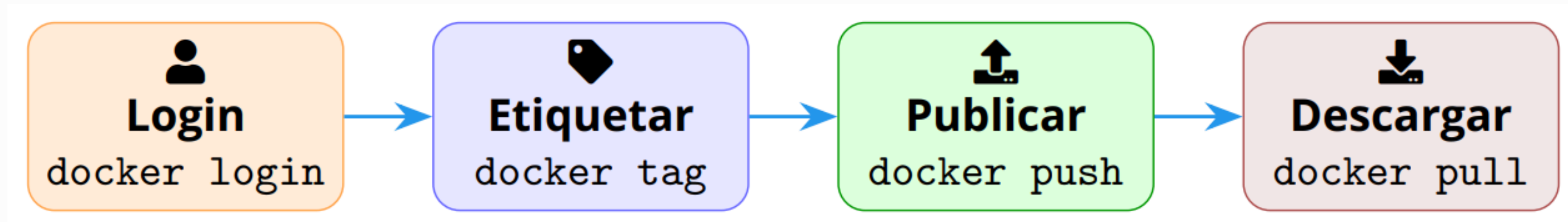
¿Cuál elegir?

- **Slim:** buena relación tamaño/compatibilidad.
- **Alpine:** máxima reducción, pero puede tener problemas con paquetes que dependen de glibc.

Para empezar, slim es la opción más segura.

Publicación en Docker Hub

FLUJO DE PUBLICACIÓN



El ciclo completo

1. Construyes tu imagen.
2. Le asignas un tag.
3. La publicas en un registro.
4. Cualquiera puede descargarla y usarla.

LOGIN Y TAGGING

1. Iniciar sesión en Docker Hub

```
docker login  
# Usuario: tu-usuario  
# Password: tu-token
```

2. Etiquetar la imagen

```
# Formato:  
# usuario/imagen:tag  
docker tag mi-flask:v1 \  
cristian/mi-flask:v1
```

Convención de tags

- v1, v2, v3: versiones específicas.
- latest: la más reciente (se actualiza manualmente).
- dev, staging, prod: por entorno.
- 2026-05-31: por fecha de build.

Importante

- latest no es automático: tú decides qué imagen lo lleva.

PUSH Y PULL

3. Publicar la imagen

```
docker push tony/mi-flask:v1  
docker push tony/mi-flask:latest
```

- La imagen queda disponible en Docker Hub según la visibilidad del repositorio.

4. Descargar y ejecutar

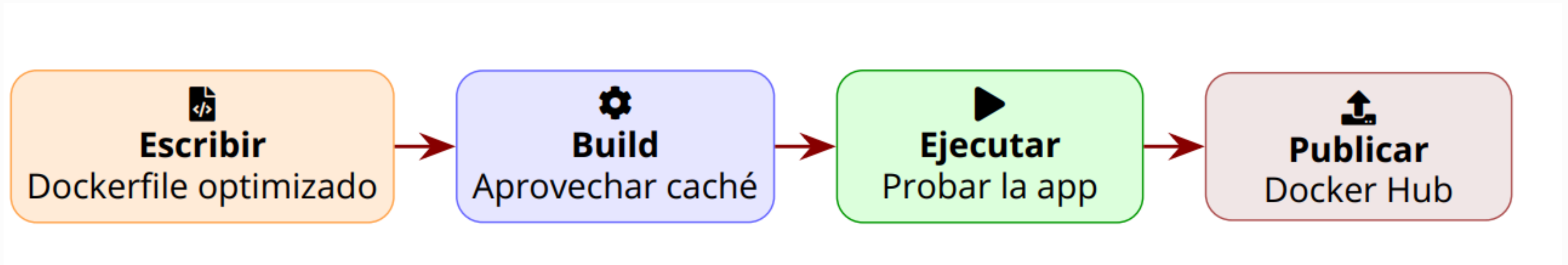
```
docker pull cristian/mi-flask:v1  
  
docker run -d -p 5000:5000 \  
cristian/mi-flask:v1
```

¿Pública o privada?

- Docker Hub permite 1 repo privado gratis.
- GHCR es alternativa para repos privados.

Laboratorio práctico

LABORATORIO: APP FLASK OPTIMIZADA



Objetivo

Partiendo del proyecto Flask de la sesión 1, optimizar el Dockerfile con buenas prácticas, construir la imagen aprovechando caché y publicarla en Docker Hub.

DOCKERFILE OPTIMIZADO PARA EL LABORATORIO

```
1 # Etapa 1: builder (si necesitas compilar )
2 FROM python :3.12 - slim AS builder
3 WORKDIR / app
4 COPY requirements . txt .
5 RUN pip install --no - cache - dir -r requirements . txt
6
7 # Etapa 2: imagen final ( runtime )
8 FROM python :3.12 - slim
9 WORKDIR / app
10 # Copiar dependencias ya instaladas desde el builder
11 COPY -- from = builder / usr / local / lib / python3 .12/ site - packages \
12 / usr / local / lib / python3 .12/ site - packages
13 COPY -- from = builder / usr / local / bin / usr / local / bin 14 COPY app . py . 15 EXPOSE 5000
16 CMD [" python ", " app .py"]
```

COMANDOS DEL LABORATORIO

1. Construir la imagen

```
docker build -t mi-flask:v2 .
```

2. Ver las capas generadas

```
docker history mi-flask:v2
```

3. Ejecutar y probar

```
docker run -d --name flask-v2 \
-p 5000:5000 mi-flask:v2
```

```
curl http://localhost:5000
```

4. Login en Docker Hub

```
docker login
```

5. Etiquetar

```
docker tag mi-flask:v2 \ cristian/mi-flask:v2
```

6. Publicar

```
docker push cristian/mi-flask:v2
```

7. Verificar en:

```
# hub.docker.com/r/cristian/mi-flask
```

ERRORES FRECUENTES EN ESTA SESIÓN

ERROR	CAUSA PROBABLE	SOLUCIÓN RÁPIDA
Build lento siempre	Mal orden de capas (COPY .. antes de RUN)	Mover dependencias antes del código
Imagen muy pesada	Usar imagen base :latest completa	Cambiar a :slim o :alpine
Caché no funciona	Cambió un archivo antes de lo esperado	Revisar orden: lo estable primero
docker push rechazado	No hay login o tag mal escrito	docker login y verificar usuario/imagen:tag
Archivos .env en la imagen	Falta .dockerignore	Crear .dockerignore y reconstruir
Error no space left	Imágenes acumuladas en disco	docker system prune -a

CHECKLIST DE APRENDIZAJE — SESIÓN 2

1. Puedo escribir un Dockerfile con FROM, RUN, COPY, WORKDIR, ENV, EXPOSE y CMD.
2. Puedo explicar la diferencia entre COPY y ADD, CMD y ENTRYPOINT.
3. Puedo explicar qué es una capa y cómo afecta el orden de las instrucciones.
4. Puedo optimizar un Dockerfile para aprovechar la caché.
5. Puedo usar .dockerignore para excluir archivos del contexto de build.
6. Puedo elegir entre imagen base completa, slim y alpine.
7. Puedo publicar una imagen en Docker Hub con tagging y versionado.
8. Puedo descargar y ejecutar una imagen desde Docker Hub.